Module 1: Applications Development

Manage application development using version control.

Estimated Time: 2 hours

NOTE: When counting from 1 to 5, we go in order: 1, 2, 3, 4, 5. Similarly, when following a step-by-step process, it's important to go through each step in sequence without skipping around. If you find yourself stuck at any point, don't hesitate to reach out to your instructor for help and support.

If a text is in orange, then it is a link. A blue text should be typed **as is**. Then all red texts should be replaced by the expected value. Just ensure that you typed in **without misspellings**.

Learning Task 3 – Creating a Git Repository

Software projects, big or small, need proper management to track progress and changes. This learning task covers tools that help simplify maintaining and scaling software projects, which are essential skills as a software developer. Think of them as good running shoes—they help you go faster and farther. A good computer and internet are required in this learning task.

Git & GitHub

Meet **git**, your go-to tool for managing projects anywhere, anytime, even on a new computer. It tracks changes, handles revisions smoothly, and enables real-time collaboration across locations (*Chacon et al.*, 2014; *Git Cheatsheet*, 2024; *Torvalds,* 2005). Now, let's install **git**!

1. Launch your browser and go to Git (https://git-scm.com/download/win) website.
    - Under the **Standalone Installer**, click the **64-bit Git for Windows Setup** link
    - Once downloaded, complete the installation.
2. Open the Command Prompt:
    - Type: `ssh-keygen -t ed25519 -C your@email.com`
    - When asked "`Enter file in which to save the key`", just hit the **ENTER** key.
    - When asked "`Overwrite (y/n)?`", type `y` and enter.
    - When asked "`Enter passphrase (empty for no passphrase):`", just hit the **ENTER** key.
    - When asked "`Enter same passphrase again:`", just hit the **ENTER** key.
    - Type: `type "C:\Users\%USERNAME%\.ssh\id_ed25519.pub"`
    - Copy the resulting output of the operations.
        i. **It starts with:** `ssh-ed25519 ...`
3. Sign in to **GitHub** website.
    - Go to **GitHub** > **Settings** > **SSH and GPG keys** (https://github.com/settings/keys)
    - Click the **New SSH key** button

© Rodney M. Maniego Jr. v2.2026

- In the **Add new SSH Key** page, enter the **DCSTPC** as the title.
- Paste your **ssh-ed25519** key generated above.
- Click the **Add SSH** key button.

4. Go back to Command Prompt. (Do not replace the `%USERNAME%` part.)
   - Open the SSH folder: `cd "C:\Users\%USERNAME%\.ssh"`
   - Create a config file: `type nul > config`
   - Open the config file: `start config`
     i. Open with Notepad/Notepad++
   - Copy the text to the **config** file.

```
Host github.com
User git
Hostname ssh.github.com
PreferredAuthentications publickey
IdentityFile ~/.ssh/id_ed25519
Port 443
```

   - This **config file** is a workaround for SSH-based Git operations (clone, push, and pull) even in restricted networks, by forcing connections to port 443 when port 22 is blocked by the network administrator.
   - Save your changes

While **git** lets you manage a project, **SSH** on the other hand is like a “secret handshake” that only you (client) and another person (server) know, so whenever you communicate, you’ll know that no one else can snoop over secure conversations. This is usually done once on every computer, once for your computers at school, at home, and elsewhere.

GitHub Repository

Git is a program used to manage your projects while **GitHub** is a service (website) where you can store your projects online. It is one of the many online services that allows its users to create, manage, and share projects with their collaborators or to the public (*Bell et al.*, 2014; *Pipinellis*, 2015). It is not only used in programming, but it is also used in fields like engineering and other applied fields (*Getting Started with Git*, 2024). While this is used practically use by real-world projects, you can also use GitHub to manage and store your mini-projects. So, your revisions remain accessible and lightweight without the need of larger disk space.

You just need to run a command to generate a pair of large numbers—your private and public keys—that are easy to create but hard to crack (*Bider*, 2018; *Silverman et al.*, 2005). Keep your **private key** secure and only share your **public key** with trusted servers like GitHub for safe communication

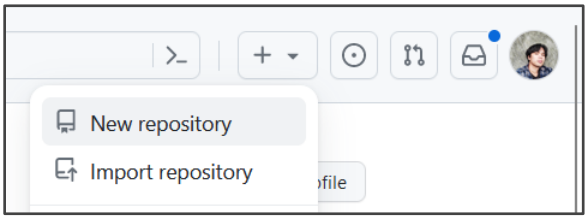
© Rodney M. Maniego Jr. v2.2026

over the internet. When you access GitHub using **git**, it will then verify your identity through a mathematical challenge based on your public key—but this happens in the background. Cool, right?

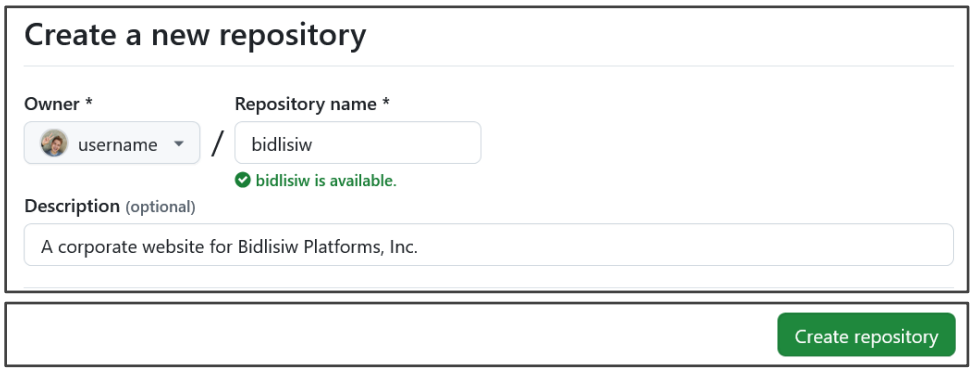
In analogy, the **internet** is a large and interconnected public highway, **GitHub** is a secure warehouse, your **computer** is your virtual office, **git** is your swift messenger, and **SSH** is your trusty bodyguard. If you have a confidential project and you need to securely transport it through a public highway all the way to a distant warehouse, you need a fast messenger and a reliable bodyguard. That is how all those things interact. Do you understand it better now?

The point here is that even if you choose to store your data online for convenience, you should remain as the sole owner of your data and you are the only person that should decide who will be able to see, read, or modify it.

1. On the top-right corner of the GitHub webpage, click the **+** icon.
    a. Click the **New Repository** option.



2. Under the Create a new repository webpage, fill up the following information.
    NOTE: Base this on your LT2 Submission
    a. **Repository name (lowercase):** company
    b. **Description:** A corporate website for Company Name.
    c. Select the **Public** option.

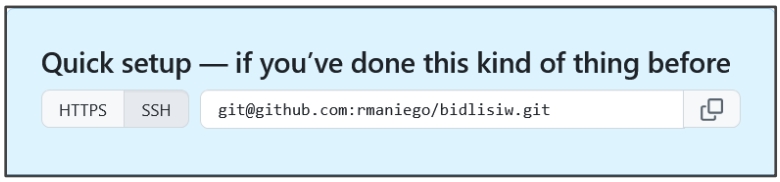


3. Click the **Create Repository** button.
4. On your new GitHub repository, click the "Settings" Settings tab.
    a. In the sidebar, find the "**Access**" action header.
    b. Click the "**Collaborators**" action item.
    c. Confirm your access using a **passkey** to continue.
    d. Under "**Manage access**" section, click the "**Add people**" button.

© Rodney M. Maniego Jr. v2.2026

   e. Enter "**rmaniego**" and click "**Add rmaniego**" button.

5. Click on the "<> Code" Code tab, then copy your **SSH** Clone URL.
   a. Use **SSH** and **not HTTPS** for this and our future activities.



Pushing Changes

For every project, create its own git repository. Then create a local *copy* in your computer using the **git clone** command. Each time you have successfully completed a task, you **add**, **commit**, and **push** all your changes to GitHub (*Pushing Commits to a Remote Repository*, 2024). What happens under-the-hood is complex, but in simpler terms, **git** takes note of the **specific changes** that you made and use this information to track it and recover older versions. Without running **add**, **commit**, and **push** your changes are just temporary and prone to loss.

1. Ensure that git is installed already.
2. Launch **Command Prompt**:
   a. Go to **Local Disk C**: `cd /d c:/`
   b. Create a new folder: `mkdir projects`
   c. Open the folder: `cd projects`
   d. Run (only required once): `git config --global user.name "Your Full Name"`
   e. Run (only required once): `git config --global user.email "email@gmail.com"`
   f. Clone your repository: `git clone git@github.com:username/company.git`
   g. When prompted, respond with the command `Y` or `y` to continue.
3. After successfully cloning the repository:
   a. Open your repository: `cd company`
   b. Create a file: `type nul > CHANGELOG.md`
   c. Open the file: `start .`
   d. Copy the following contents to the file.

```
# Changelog

All notable changes to this project will be documented in this file.
This project follows [Semantic Versioning](https://semver.org/).

## [1.0.0] - yyyy-mm-dd
### Added
- New `CHANGELOG.md` file for tracking purposes.
```

4. After updating the file, push your changes to the remote repository.

© Rodney M. Maniego Jr. v2.2026

   a. Stage all changes: `git add .`
   b. Commit all staged changes: `git commit -m "[web] Add project to GitHub"`
   c. Push commits to GitHub: `git push -u origin main`
   d. Screenshot the outputs in the Command Line (for email attachments).
   e. Clear the screen: `cls`
5. Create new branches in your repository.
   a. Create the **dev** branch: `git switch -c dev`
   b. Push the **dev** branch: `git push origin dev`
   c. Create the **beta** branch: `git switch -c beta`
   d. Push the **beta** branch: `git push origin beta`
   e. Create the **stable** branch: `git switch -c stable`
   f. Push the **stable** branch: `git push origin stable`
   g. Switch back to **main** branch: `git switch main`
   h. Screenshot the outputs in the Command Line (for email attachments).
   i. Clear the screen: `cls`
6. Once done, submit it to your instructor via email.
7. Submit **within two weeks** after the posting of this activity.
   a. Attach the screenshots in the email.

To: **rodney.maniego.jr@vsu.edu.ph**
Subject: [2025-mm-dd] CSci 23 LT3 Surname

Sir Rodney,

Good day!

I am Juan D. Cruz (student #) of your CSci 23 class. Attached are the screenshots for your reference. Below is the link to my GitHub repository.

https://github.com/username/company

Best regards,

Juan D. Cruz
BSCS-2

8. For any academic- and course-related difficulties, please do not hesitate to reach out anytime, so we can resolve them as early as possible.

© Rodney M. Maniego Jr. v2.2026

Summary

In this Learning Task, you learned about the relationship between the internet, your computer, GitHub, git, and SSH. Using these tools, you can successfully create and manage projects like a pro! Next time, we will use this learned knowledge and skills when creating new projects.

References

Bell, P., & Beer, B. (2014). *Introducing GitHub: A Non-Technical Guide*. O'Reilly Media, Inc.

Bider, D. (2018). Use of RSA Keys with SHA-256 and SHA-512 in the Secure Shell (SSH) Protocol (Request for Comments RFC 8332). Internet Engineering Task Force. https://doi.org/10.17487/RFC8332

Chacon, S., & Straub, B. (2014). Git on the server – Setting up the server. In Pro Git (2nd ed.). Apress. ISBN 978-1484200773.

Pipinellis, A. (2015). *GitHub Essentials*. Packt Publishing.

Silverman, E., & Byrnes, R. (2005). SSH: The Secure Shell (The Definitive Guide) (2nd ed.). O'Reilly.

Torvalds, L. (2005). Initial revision of "git", the information manager from hell [Commit message]. GitHub. https://github.com/git/git/commit/e83c5163316f89bfbde7d9ab23ca2e25604af290

Getting started with Git. (2024). GitHub Docs. Retrieved April 8, 2024, from https://docs.github.com/en/get-started/getting-started-with-git

Git Cheatsheet. (2024). GitHub Docs. Retrieved April 8, 2024, from https://docs.github.com/en/get-started/getting-started-with-git/git-cheatsheet

© Rodney M. Maniego Jr. v2.2026